

LSTM

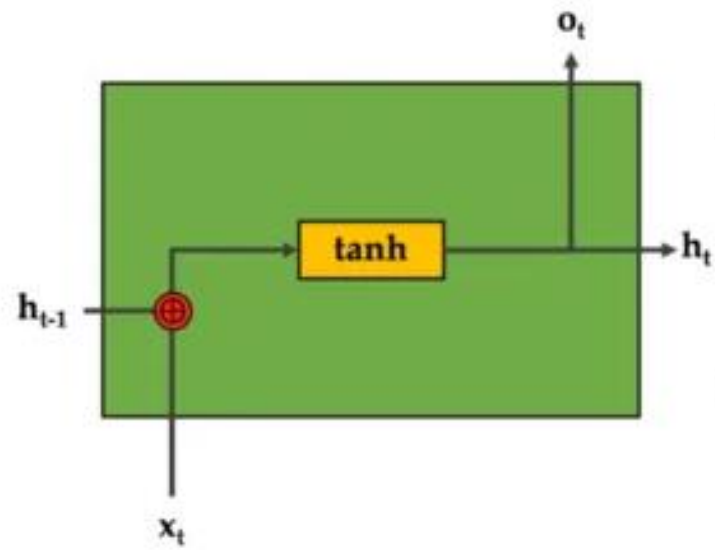
Motivation

“Ayesha went to the university because she wanted to study.”

When the model reaches **“she”**, it needs to remember **Ayesha**.

a **cell state** carries longer-term memory, while the **hidden state** carries the current representation/output of the **LSTM cell**.

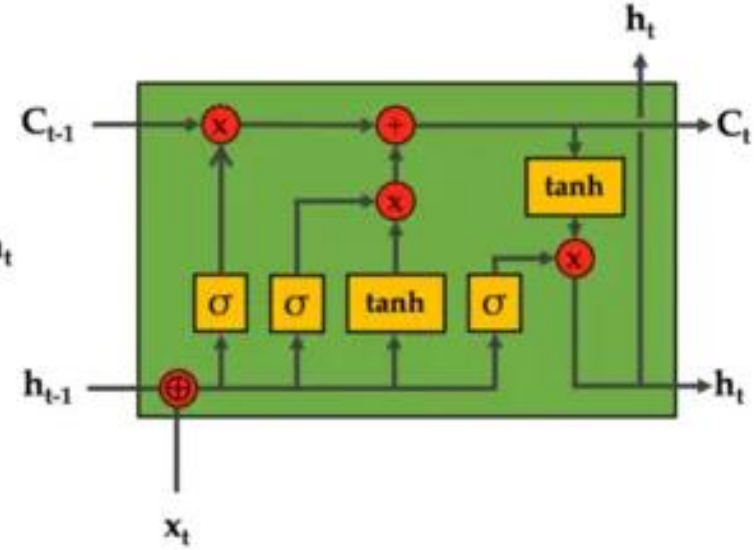
RNN



σ sigmoid function

$\oplus$ pointwise addition

LSTM



tanh hyperbolic tangent function

$\otimes$ pointwise multiplication

Long Short-Term Memory (LSTM)

- A type of RNN proposed by Hochreiter and Schmidhuber in 1997 as a solution to the vanishing gradients problem.
- On step t , there is a **hidden state** $h^{(t)}$ and a **cell state** $c^{(t)}$
 - Both are vectors length n
 - The cell stores **long-term information**
 - The LSTM can **erase**, **write** and **read** information from the cell
- The selection of which information is **erased/written/read** is controlled by three corresponding **gates**
 - The gates are also vectors length n
 - On each timestep, each element of the gates can be **open** (1), **closed** (0), or somewhere in-between.
 - The gates are **dynamic**: their value is computed based on the current context

Long Short-Term Memory (LSTM)

We have a sequence of inputs $\mathbf{x}^{(t)}$, and we will compute a sequence of hidden states $\mathbf{h}^{(t)}$ and cell states $\mathbf{c}^{(t)}$. On timestep t :

Forget gate: controls what is kept vs forgotten, from previous cell state

Input gate: controls what parts of the new cell content are written to cell

Output gate: controls what parts of cell are output to hidden state

New cell content: this is the new content to be written to the cell

Cell state: erase ("forget") some content from last cell state, and write ("input") some new cell content

Hidden state: read ("output") some content from the cell

Sigmoid function: all gate values are between 0 and 1

$$\mathbf{f}^{(t)} = \sigma \left(\mathbf{W}_f \mathbf{h}^{(t-1)} + \mathbf{U}_f \mathbf{x}^{(t)} + \mathbf{b}_f \right)$$

$$\mathbf{i}^{(t)} = \sigma \left(\mathbf{W}_i \mathbf{h}^{(t-1)} + \mathbf{U}_i \mathbf{x}^{(t)} + \mathbf{b}_i \right)$$

$$\mathbf{o}^{(t)} = \sigma \left(\mathbf{W}_o \mathbf{h}^{(t-1)} + \mathbf{U}_o \mathbf{x}^{(t)} + \mathbf{b}_o \right)$$

$$\tilde{\mathbf{c}}^{(t)} = \tanh \left(\mathbf{W}_c \mathbf{h}^{(t-1)} + \mathbf{U}_c \mathbf{x}^{(t)} + \mathbf{b}_c \right)$$

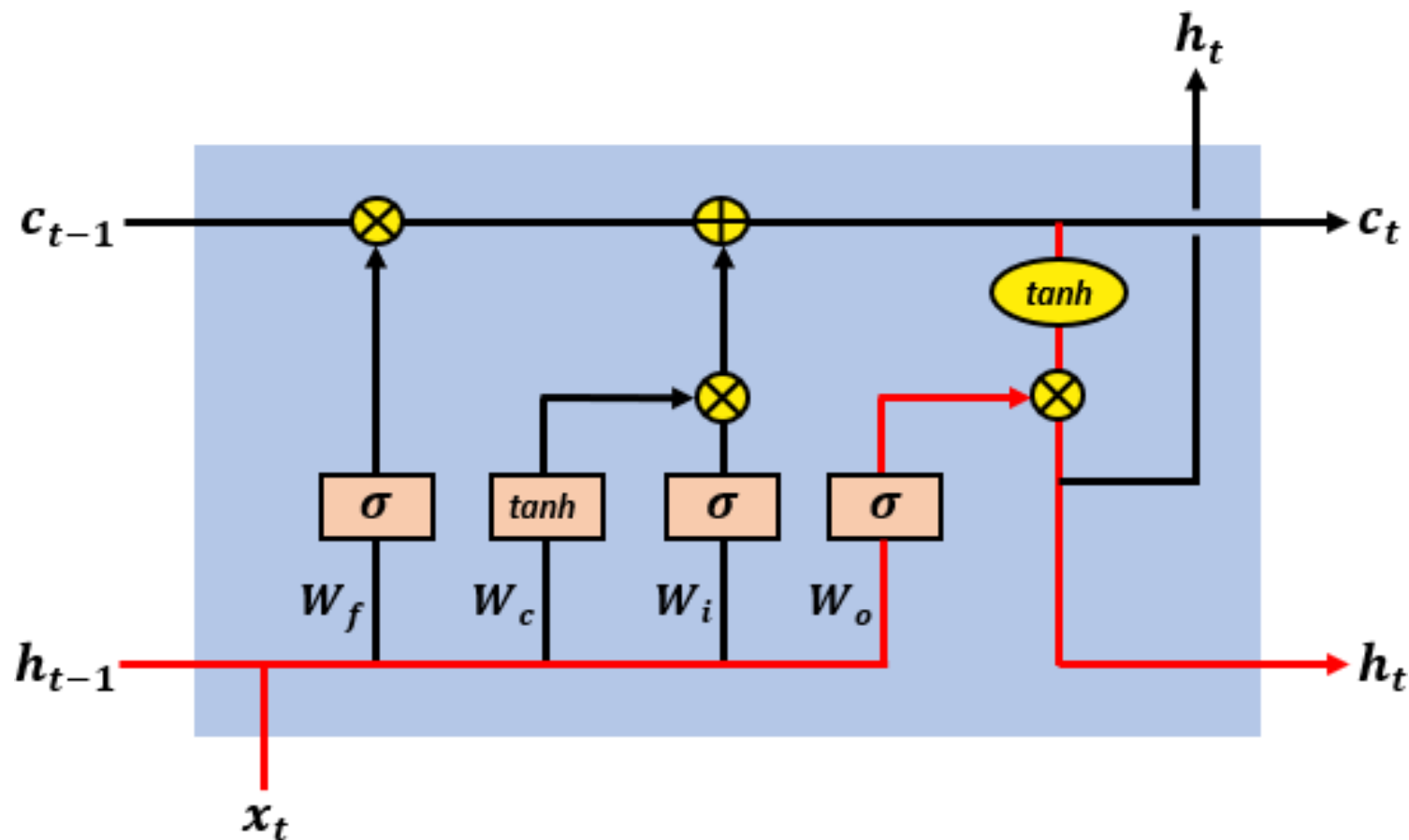
$$\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \circ \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \circ \tilde{\mathbf{c}}^{(t)}$$

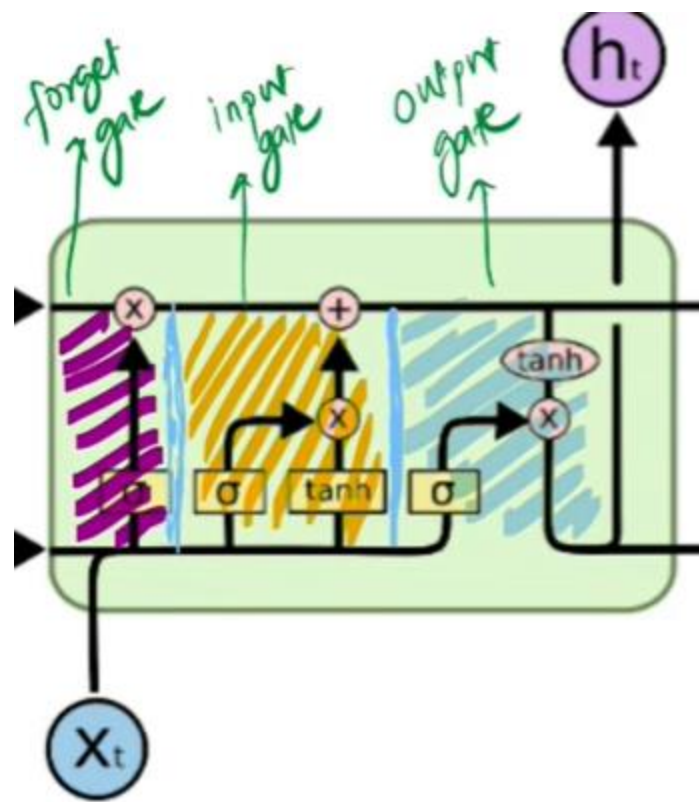
$$\mathbf{h}^{(t)} = \mathbf{o}^{(t)} \circ \tanh \mathbf{c}^{(t)}$$

Gates are applied using element-wise product

All these are vectors of same length n

The LSTM output gate's action on the cell state





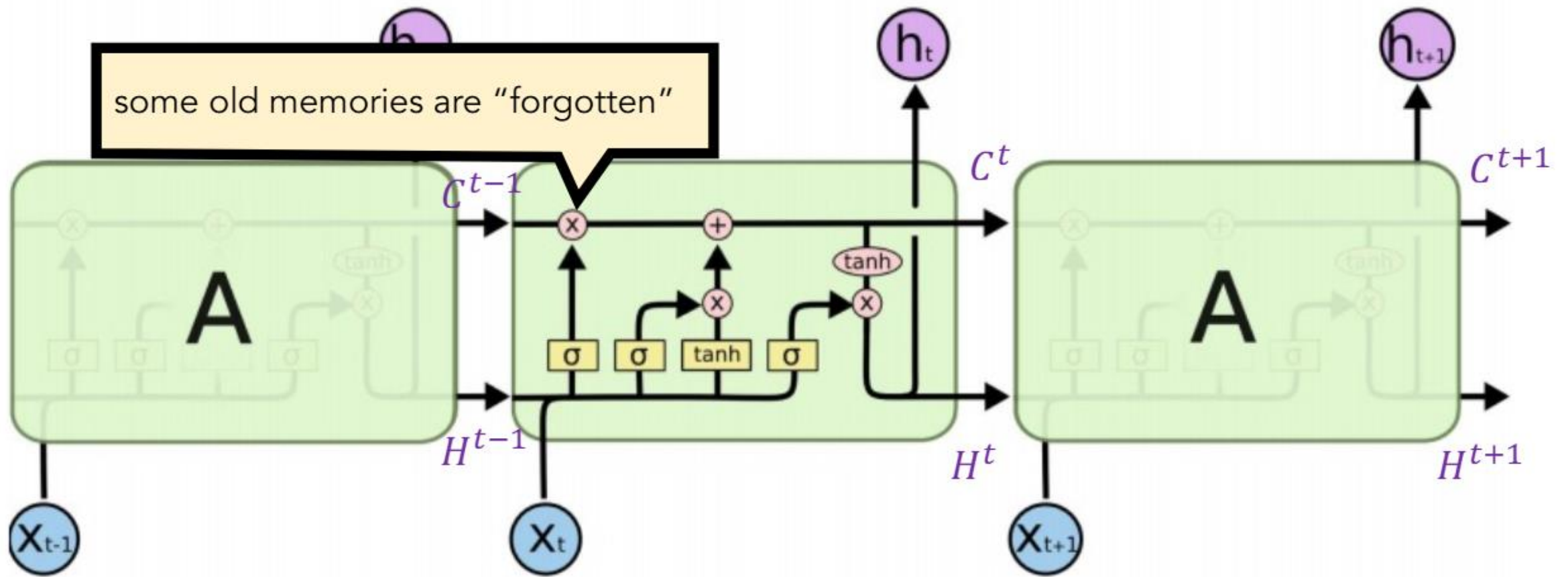
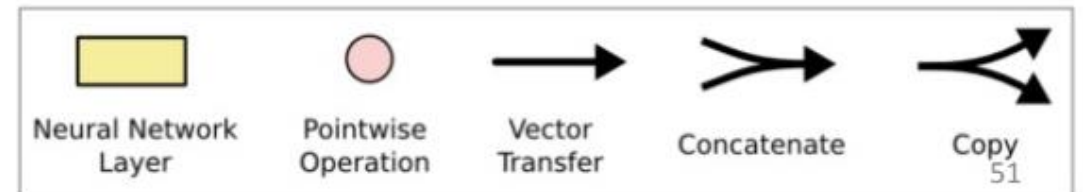


Diagram: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>



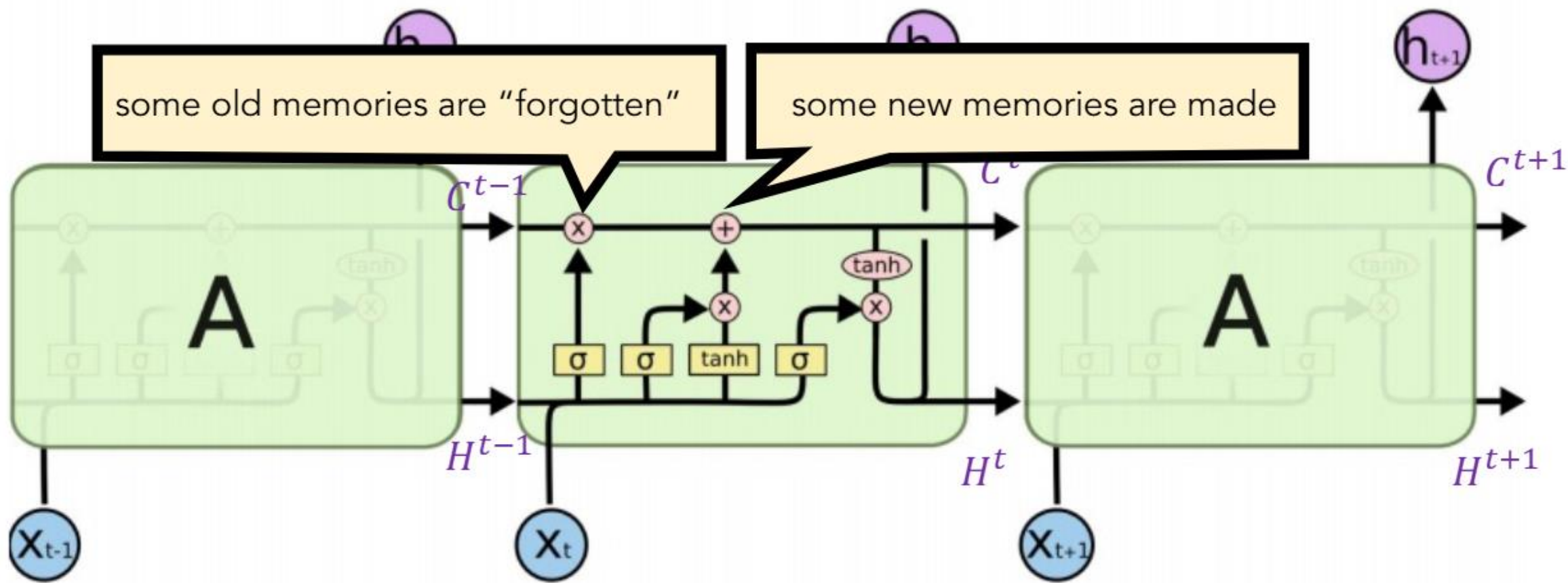
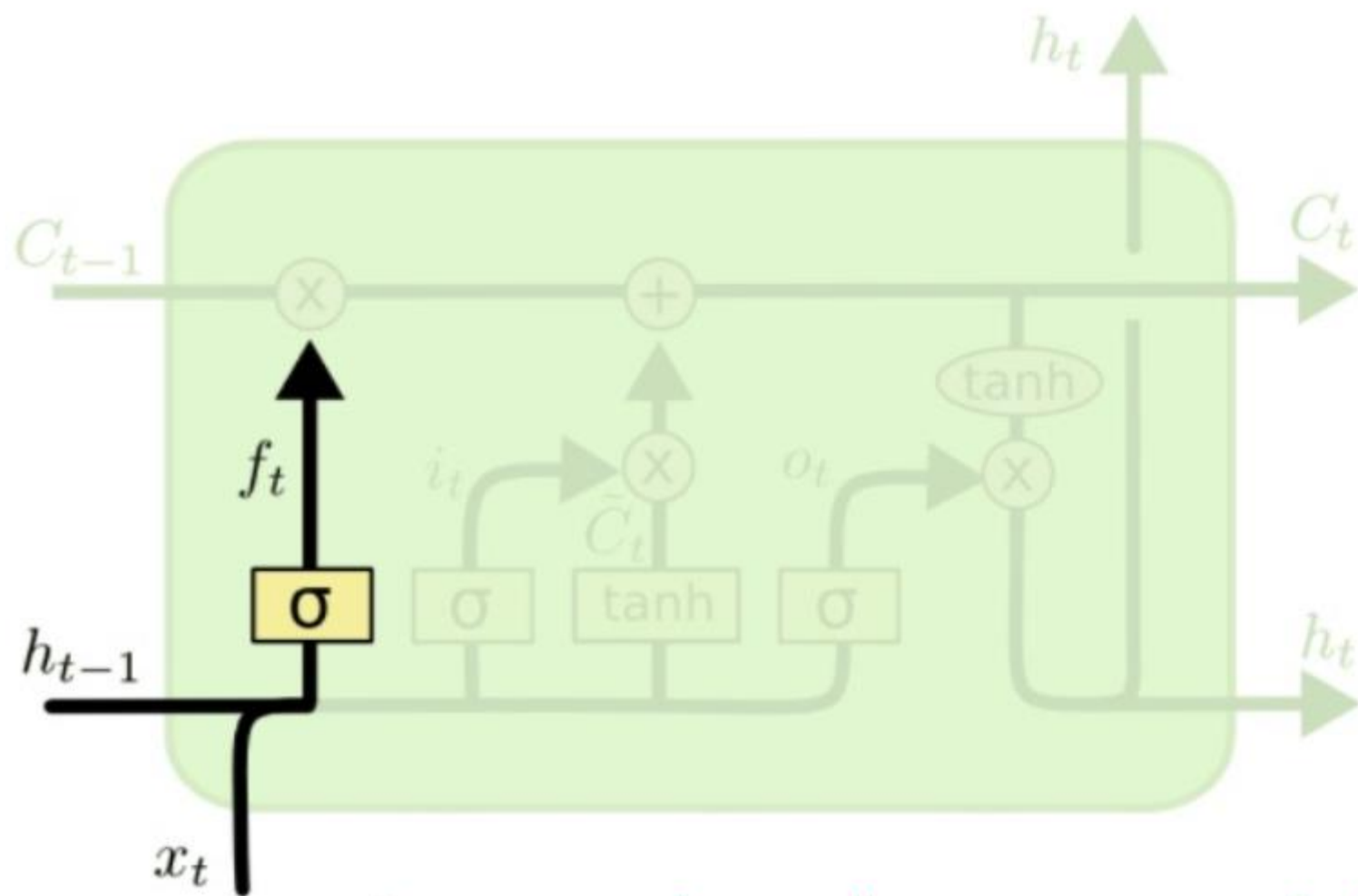
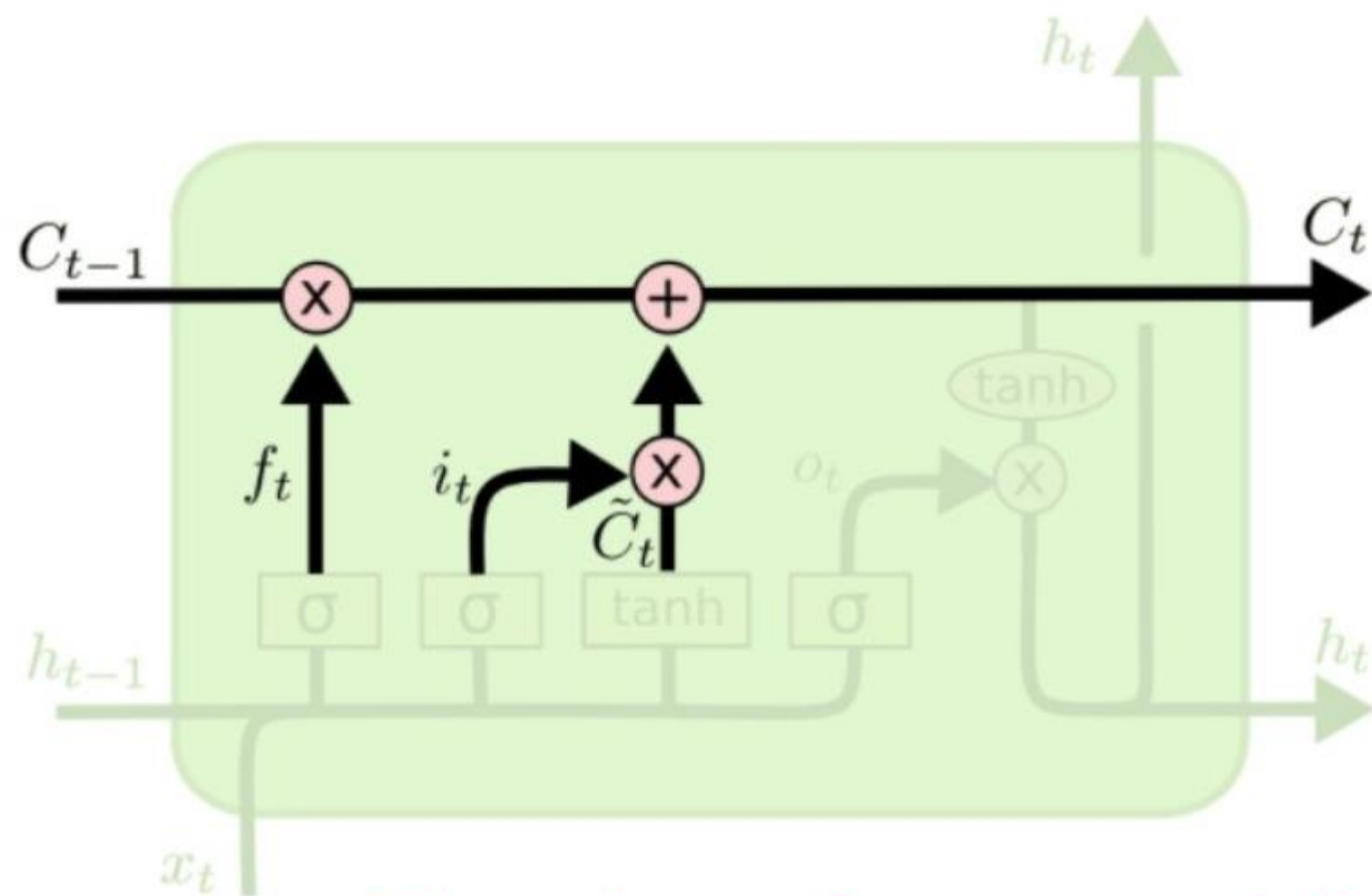


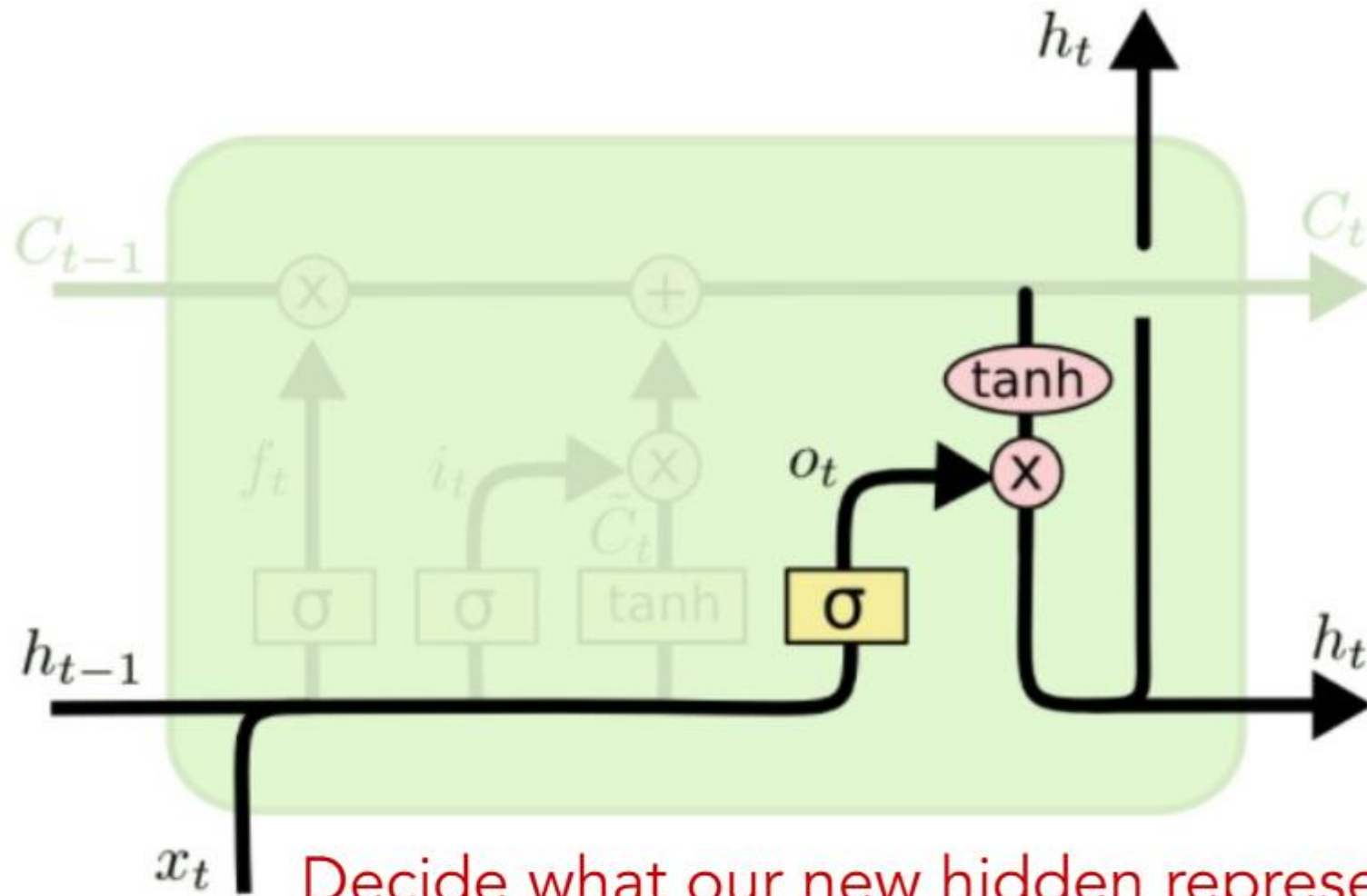
Diagram: <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>



Imagine the cell state currently has values related to a previous topic. Convo has shifted.



The cell state forgets some info, then it's simultaneously updated by us adding to it.

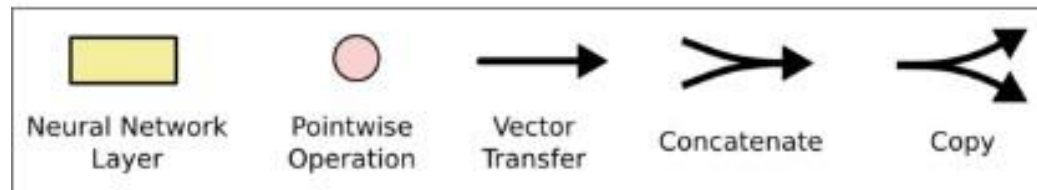
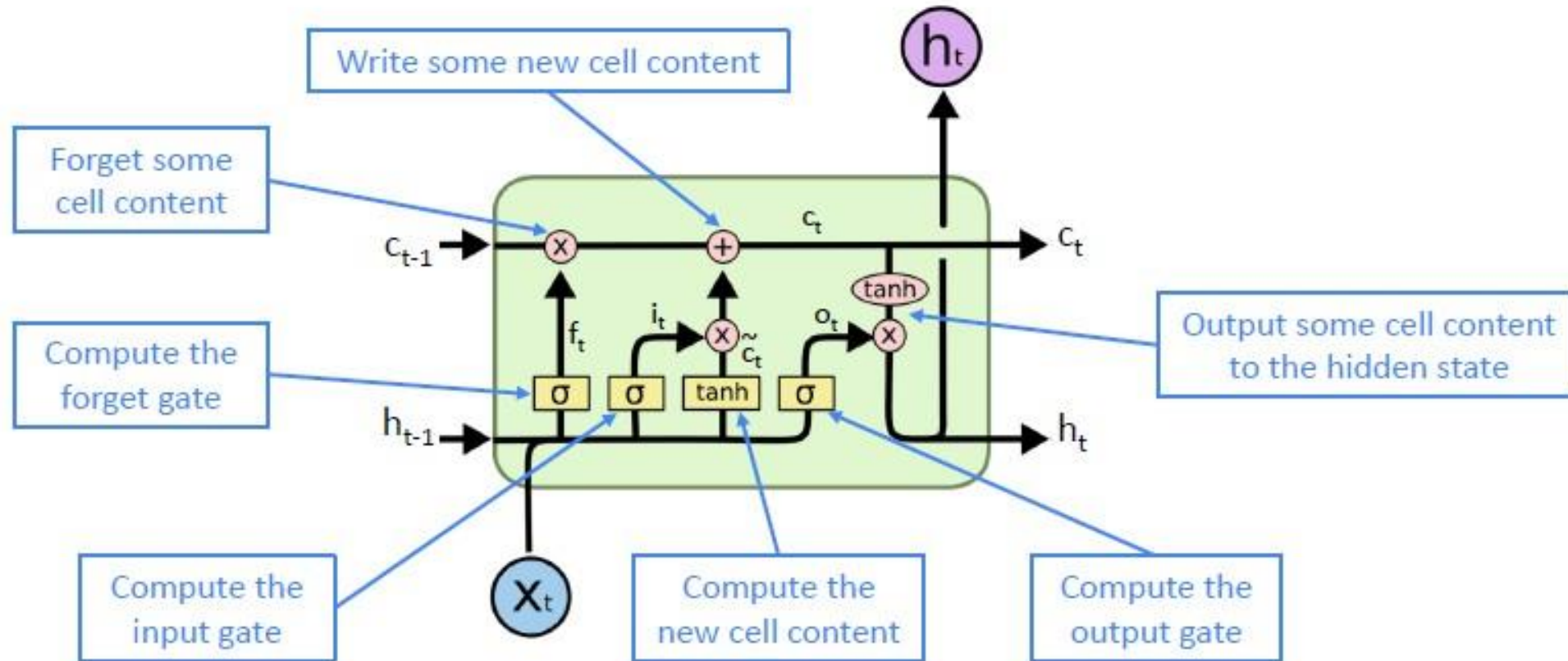


Decide what our new hidden representation will be. Based on:

- filtered version of the cell state, and
- weighted version of recurrent, hidden layer

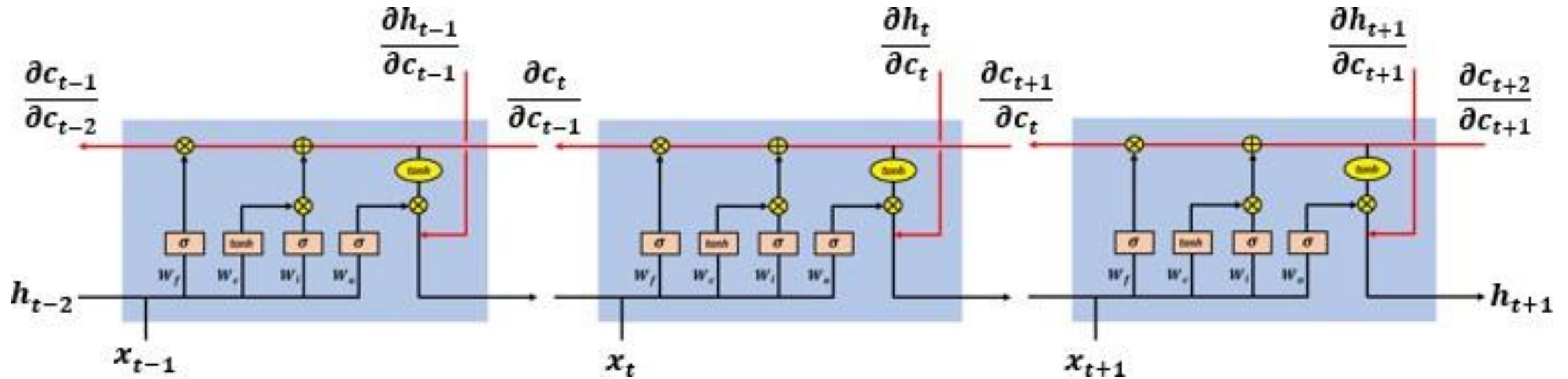
Long Short-Term Memory (LSTM)

You can think of the LSTM equations visually like this:



How LSTM Solves Vanishing Gradient Problem

Back propagating through time for gradient computation



The gradient of the error for some time step k has the form:

$$\begin{aligned}\frac{\partial E_k}{\partial W} &= \frac{\partial E_k}{\partial h_k} \frac{\partial h_k}{\partial c_k} \dots \frac{\partial c_2}{\partial c_1} \frac{\partial c_1}{\partial W} \\ &= \frac{\partial E_k}{\partial h_k} \frac{\partial h_k}{\partial c_k} \left(\prod_{t=2}^k \frac{\partial c_t}{\partial c_{t-1}} \right) \frac{\partial c_1}{\partial W} \quad (4)\end{aligned}$$

the following product causes the gradients to vanish:

$$\prod_{t=2}^k \frac{\partial c_t}{\partial c_{t-1}}$$

$$c_t = c_{t-1} \otimes f_t \oplus \tilde{c}_t \otimes i_t \quad (5)$$

$$\begin{aligned} \frac{\partial c_t}{\partial c_{t-1}} &= \frac{\partial}{\partial c_{t-1}} [c_{t-1} \otimes f_t \oplus \tilde{c}_t \otimes i_t] \\ &= \frac{\partial}{\partial c_{t-1}} [c_{t-1} \otimes f_t] + \frac{\partial}{\partial c_{t-1}} [\tilde{c}_t \otimes i_t] \\ &= \frac{\partial f_t}{\partial c_{t-1}} \cdot c_{t-1} + \frac{\partial c_{t-1}}{\partial c_{t-1}} \cdot f_t + \frac{\partial i_t}{\partial c_{t-1}} \cdot \tilde{c}_t + \frac{\partial \tilde{c}_t}{\partial c_{t-1}} \cdot i_t \end{aligned}$$

$$\frac{\partial c_t}{\partial c_{t-1}} = \sigma'(W_f \cdot [h_{t-1}, x_t]) \cdot W_f \cdot o_{t-1} \otimes \tanh'(c_{t-1}) \cdot c_{t-1}$$

$$+ f_t$$

$$+ \sigma'(W_i \cdot [h_{t-1}, x_t]) \cdot W_i \cdot o_{t-1} \otimes \tanh'(c_{t-1}) \cdot \tilde{c}_t$$

$$+ \sigma'(W_c \cdot [h_{t-1}, x_t]) \cdot W_c \cdot o_{t-1} \otimes \tanh'(c_{t-1}) \cdot i_t$$

$$A_t = \sigma'(W_f \cdot [h_{t-1}, x_t]) \cdot W_f \cdot o_{t-1} \otimes \tanh'(c_{t-1}) \cdot c_{t-1}$$

$$B_t = f_t$$

$$C_t = \sigma'(W_i \cdot [h_{t-1}, x_t]) \cdot W_i \cdot o_{t-1} \otimes \tanh'(c_{t-1}) \cdot \tilde{c}_t$$

$$D_t = \sigma'(W_c \cdot [h_{t-1}, x_t]) \cdot W_c \cdot o_{t-1} \otimes \tanh'(c_{t-1}) \cdot i_t$$

We know:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Apply the chain rule:

$$\frac{\partial f_t}{\partial C_{t-1}} = \sigma'(\dots) \cdot W_f \cdot \frac{\partial h_{t-1}}{\partial C_{t-1}}$$

Now:

$$h_{t-1} = o_{t-1} \odot \tanh(C_{t-1})$$

So:

$$\frac{\partial h_{t-1}}{\partial C_{t-1}} = o_{t-1} \odot \tanh'(C_{t-1})$$

Therefore:

$$\frac{\partial f_t}{\partial C_{t-1}} = \sigma'(W_f[h_{t-1}, x_t]) W_f (o_{t-1} \odot \tanh'(C_{t-1}))$$

Then remember our product-rule equation has:

$$\frac{\partial f_t}{\partial C_{t-1}} C_{t-1}$$

So we get:

$$\sigma'(W_f[h_{t-1}, x_t]) W_f (o_{t-1} \odot \tanh'(C_{t-1})) C_{t-1}$$

$$\frac{\partial c_t}{\partial c_{t-1}} = A_t + B_t + C_t + D_t \quad (6)$$

$$\frac{\partial E_k}{\partial W} = \frac{\partial E_k}{\partial h_k} \frac{\partial h_k}{\partial c_k} \left(\prod_{t=2}^k [A_t + B_t + C_t + D_t] \right) \frac{\partial c_1}{\partial W}$$

Preventing the error gradients from vanishing

Notice that the gradient contains the forget gate's vector of activations, which allows the network to better control the gradients values, at each time step, using suitable parameter updates of the forget gate.

$$\frac{\partial c_t}{\partial c_{t-1}} = \sigma'(W_f \cdot [h_{t-1}, x_t]) \cdot W_f \cdot o_{t-1} \otimes \tanh'(c_{t-1}) \cdot c_{t-1}$$

$$+ f_t$$

$$+ \sigma'(W_i \cdot [h_{t-1}, x_t]) \cdot W_i \cdot o_{t-1} \otimes \tanh'(c_{t-1}) \cdot \tilde{c}_t$$

$$+ \sigma'(W_c \cdot [h_{t-1}, x_t]) \cdot W_c \cdot o_{t-1} \otimes \tanh'(c_{t-1}) \cdot i_t$$

Say that for some time step $k < T$, we have that:

$$\sum_{t=1}^k \frac{\partial E_t}{\partial W} \rightarrow 0$$

Then for the gradient not to vanish, we can find a suitable parameter update of the forget gate at time step $k+1$ such that:

$$\frac{\partial E_{k+1}}{\partial W} \nrightarrow 0$$

It is the presence of the forget gate's vector of activations in the gradient term along with additive structure which allows the LSTM to find such a parameter update at any time step, and this yields:

$$\sum_{t=1}^{k+1} \frac{\partial E_t}{\partial W} \nrightarrow 0$$

Another important property to notice is that the cell state gradient is an additive function

$$A_t \approx \overrightarrow{0.1}, B_t = f_t \approx \overrightarrow{0.7}, C_t \approx \overrightarrow{0.1}, D_t \approx \overrightarrow{0.1}$$

$$\begin{aligned}\prod_{t=2}^k [A_t + B_t + C_t + D_t] &\approx \prod_{t=2}^k [\overrightarrow{0.1} + \overrightarrow{0.7} + \overrightarrow{0.1} + \overrightarrow{0.1}] \\ &\approx \prod_{t=2}^k \overrightarrow{1} \nrightarrow 0\end{aligned}$$

$$\frac{\partial E}{\partial W} = \sum_{t=1}^T \frac{\partial E_t}{\partial W} \quad (3)$$

In **RNNs**, the sum in (3) is made from expressions with a similar behavior that are likely to all be in $[0,1]$ which causes vanishing gradients.

In **LSTMs**, however, the presence of the forget gate, along with the additive property of the cell state gradients, enables the network to update the parameter in such a way that the different sub gradients in (3) do not necessarily agree

$$\sum_{t=1}^T \frac{\partial E_t}{\partial W} \nrightarrow 0$$

Summary

- We have seen that RNNs suffer from vanishing gradients and caused by long series of multiplications of small values, diminishing the gradients and causing the learning process to become degenerate. In a analogous way, RNNs suffer from exploding gradients affected from large gradient values and hampering the learning process.
- LSTMs solve the problem using a unique additive gradient structure that includes direct access to the forget gate's activations, enabling the network to encourage desired behavior from the error gradient using frequent gates update on every time step of the learning process.

Is vanishing/exploding gradient just a RNN problem?

- No! It can be a problem for all neural architectures (including feed-forward and convolutional), especially deep ones.
 - Due to chain rule / choice of nonlinearity function, gradient can become vanishingly small as it backpropagates
 - Thus lower layers are learnt very slowly (hard to train)
 - Solution: lots of new deep feedforward/convolutional architectures that add more direct connections (thus allowing the gradient to flow)
- Conclusion: Though vanishing/exploding gradients are a general problem, RNNs are particularly unstable due to the repeated multiplication by the same weight matrix [Bengio et al, 1994]

LSTMs: real-world success

- In 2013-2015, LSTMs started achieving state-of-the-art results
- Successful tasks include: handwriting recognition, speech recognition, machine translation, parsing, image captioning
- LSTM became the dominant approach

Help material

- <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>
- [How LSTM networks solve the problem of vanishing gradients | by Nir Arbel | DataDrivenInvestor](#)
- <https://www.cse.iitm.ac.in/~miteshk/CS7015/Slides/Handout/Lecture15.pdf>